

DEMO PRESENTATION

실시간 서울시 버스 3D GIS 시각화 서비스

MapLibre GL JS 3D 지형 지도 위에 three.js로 실시간 버스 위치를 3D
모델로 시각화하는 토이 프로젝트

MapLibre GL JS

three.js

Node.js / Express

Vanilla JS

서울시 공공데이터 API

2026 · Claude Code와 함께 만든 토이 프로젝트

1. 주제

지도 위에 실제로 움직이는 사물을 3D로 자연스럽게 표현하는 것 — 실시간 공공데이터(서울시 버스 위치)를 엮어 "실제로 쓸모 있는" 데모로 완성했습니다.

무엇을 만들었나

MapLibre GL JS로 3D 지형(terrain) 지도를 띄우고, 그 위에 three.js 커스텀 레이어로 3D 버스 모델을 올립니다. 서울시 버스 실시간 위치 공공 API에서 특정 노선의 버스 좌표를 주기적으로 가져와, API 응답에 들어있는 버스 수만큼 3D 모델을 자동으로 생성/갱신/삭제합니다. 오른쪽 드롭다운에서 노선을 고르면 그 노선의 실시간 버스 위치로 화면이 전환됩니다.

핵심 아이디어

GIS 지형 인지 배치

지도가 3D 지형(terrain)을 켜고 있어, 버스가 항상 해당 좌표의 실제 지표면 고도 위에 올라가도록 매 프레임 지형 고도를 조회해 보정합니다.

실시간 자연스러운 이동

API는 5~10초에 한 번만 새 좌표를 주지만, 화면에서는 그 시간 동안 부드럽게 이동하는 것처럼 보이도록 좌표를 보간(interpolation)해서 렌더링합니다.

UX 정보 패널 + 추적

오른쪽 패널에서 노선 선택, 버스별 이동/정지 상태, 좌표를 확인하고, 행을 클릭하면 그 버스를 카메라가 쫓아다니면서 계속 따라가도록 했습니다.

최적화 정밀도/성능 문제 해결

버스가 여러 대일 때 렌더링이 찌그러지는 문제를 겪었고, 원인(부동소수점 정밀도)을 찾아 단일 모델과 동일한 방식으로 다시 설계해 해결했습니다.

2. 기술 스택

영역	기술	역할
서버	Node.js + Express	정적 파일 서빙(<code>src/</code> , <code>public/</code>) + 서울시 버스 API 프록시(<code>/api/bus-position</code>). 서비스키는 서버에만 보관하고 브라우저에는 노출하지 않음
지도	MapLibre GL JS	3D 지형(raster-dem terrain), 라스터 베이스맵, 커스텀 레이어(three.js) 호스팅
3D 렌더링	three.js	OBJ/MTL 모델 로딩, 커스텀 카메라 투영행렬 기법으로 mercator 좌표계 위에 모델 배치, OutlinePass 기반 마우스오버 하이라이트
프론트엔드	Vanilla JS (ES Modules)	프레임워크·번들러 없이 <code><script type="module"></code> + importmap으로 three.js/maplibre-gl을 CDN에서 직접 로드
빌드	esbuild	배포용 빌드 시에만 번들링/최소화/트랜스파일 (개발 중에는 미사용, 소스 그대로 서빙)
데이터	서울시 버스 실시간 위치 API (공공데이터포털)	노선ID로 해당 노선의 실시간 버스 좌표(WGS84), 정류소 정차 여부 등을 XML로 응답

왜 프레임워크 없이 Vanilla JS인가 — 토이 프로젝트 성격에 맞춰 빌드 도구 의존성을 최소화하고, MapLibre/three.js의 저수준 API(커스텀 레이어, 카메라 투영행렬 직접 제어)를 다루는 실험에 집중하기 위해 React 등 프레임워크를 배제했습니다.

3. 코드 구성

서버

server.js

사용 중

Express 서버. `src/`와 `public/`을 같은 루트로 정적 서빙하고, `/api` 라우터를 마운트합니다.

router.js

사용 중

서울시 버스 위치 API(`ws.bus.go.kr`)를 프록시하는 `/api/bus-position` 엔드포인트. 서비스키 (`SEOUL_BUS_API_KEY`)는 서버 환경변수로만 보관합니다.

프론트엔드 — 핵심

src/app.js

사용 중

지도 초기화, 노선 드롭다운(`public/json/bus_routeid.json` 로드), API 폴링 시작/전환, 오른쪽 버스 정보 패널 렌더링, 클릭한 버스를 따라가는 카메라 팔로우(줌 애니메이션 포함), 좌측 상단 좌표/FPS/컴퍼스 표시를 담당하는 최상위 스크립트입니다.

src/component/liveBusFleet.js

사용 중

실시간 API 데이터로 버스를 그리는 핵심 컴포넌트. `plainNo` (차량 번호판)를 키로 버스별 3D 모델/라벨을 관리하며, 새 좌표가 오면 그 자리로 순간이동하지 않고 API 호출 주기에 맞춰 보간 이동시킵니다. 버스마다 자기 전용 카메라 투영행렬을 만들어 한 대씩 순서대로 그리는 방식으로, 여러 대를 그릴 때 생기던 렌더링 정밀도 문제를 해결했습니다.

src/mixin/api.js

사용 중

`BusApiMixin` — 컴포넌트에 명시적으로 섞어 써야만 API 폴링이 동작하는 믹스인. XML 응답을 파싱해 차량번호/좌표(tmX, tmY)/정차 여부 등을 추출합니다.

src/base/BaseModel.js

사용 중

모든 3D 모델 레이어가 상속하는 공통 베이스 클래스. 카메라/씬/조명/렌더러 초기화, 좌표 변환 행렬 계산, 라벨 위치 계산, 캔버스 리사이즈 대응을 공통으로 처리합니다.

src/component/modelLabel.js

사용 중

버스 위에 떠 있는 DOM 라벨. 차량 번호와 이동/정지 상태를 나타내는 점(녹색/빨간색)을 함께 표시합니다.

src/effect/hoverOutline.js

사용 중

마우스오버 시 파란색 외곽선을 그리는 `OutlinePass` 기반 헬퍼. 여러 커스텀 레이어가 캔버스 하나를 공유하는 구조라, 외곽선만 덧셈 블렌딩으로 화면에 얹는 방식으로 다른 레이어를 가리지 않게 만들었습니다.

프론트엔드 — 이전 버전 (현재 미사용, 참고용)

파일	설명
<code>bus_g7.js</code>	지도 클릭으로 목적지를 지정해 이동하던 초기 단일 버스 버전
<code>busFleet.js</code>	기준 위치 주변을 랜덤하게 돌아다니며 서로 충돌을 피하던 가상 버스 편대
<code>routePointIcons.js</code>	미리 정의한 경로 지점마다 순번 아이콘을 표시하던 컴포넌트
<code>carm.js</code> / <code>carFleet.js</code>	버스 대신 자동차 모델로 실험하던 버전
<code>airplain.js</code> / <code>airport_1terminal.js</code>	InstancedMesh 비행기 편대 등 초기 렌더링 기법 실험

4. 프롬프트 진행 요약

Claude Code와의 대화로 진행한 개발 과정을 시간 순서로 요약합니다. 문제 발생 → 원인 진단 → 재설계의 흐름이 반복적으로 나타납니다.

1) 3D 모델 교체 실험 — 자동차 → 버스

버스를 자동차 모델로 바꾸고, 여러 대를 동시에 랜덤하게 움직이는 "편대"를 만들어 달라는 요청으로 시작. 이후 다시 버스 모델로 전환하며 같은 구조를 재사용.

2) 렌더링 깨짐 문제 진단

여러 대를 `InstancedMesh` 로 한 번에 그렸더니 모델이 찌그러지거나 카메라 각도에 따라 사라지는 문제 발생. 원인을 좁혀가며 "GPU에 올라가는 좌표의 부동소수점 정밀도 손실"임을 특정하고, 단일 모델과 동일하게 카메라 투영 행렬에 좌표를 미리 곱해 넣는 방식으로 전면 재설계.

3) 움직임 자연스럽게 다듬기

회전 방향 부호 오류, 목표 지점 도착 후 멈춰 있는 문제, 정지 후 자동 재출발 등을 순차적으로 요청받아 수정. 편대 간 충돌 회피 (separation steering) 로직도 추가.

4) 정보 패널 + 카메라 추적 UI 추가

오른쪽에 버스 목록을 표로 보여주는 패널을 만들고, 행을 클릭하면 해당 버스를 줌인해서 카메라가 따라가도록 구현. 이동/정지 상태를 점 색으로 표시.

5) 실제 공공 API 연동으로 전환

가상의 랜덤 이동 버스 대신, 서울시 버스 실시간 위치 API 데이터로 완전히 교체. API 응답 개수만큼 버스를 동적으로 생성/삭제하고, 차량번호(plainNo)를 키로 기존 버스는 재사용하도록 재설계. 5~10초 폴링 주기에 맞춰 위치를 부드럽게 보간 이동시키는 기능 추가.

6) 노선 선택 드롭다운 추가

노선명/ID 목록 JSON 파일을 읽어 드롭다운을 채우고, 선택한 노선으로 실시간 조회를 전환하는 기능 추가. 이 과정에서 잘못된 형식이던 JSON 데이터 파일도 함께 수정.

7) 세부 튜닝

API 폴링 주기, 이동 애니메이션 시간, 이동/정지 판정 기준 (API 필드 대신 실제 좌표 이동 거리로 계산) 등을 반복적으로 조정하며 완성도를 높임.

진행 방식의 특징 — 단순히 기능을 하나씩 추가하기보다, 렌더링 버그가 발생할 때마다 "왜 깨지는가"를 함께 진단하고 원인에 맞는 구조로 다시 설계하는 과정을 반복했습니다. 특히 정밀도 문제(3번)는 여러 차례의 시행착오 끝에 근본 원인을 찾아 해결한 사례입니다.